

Análisis de sentimiento

PREDICCIÓN ELECCIONES 2024

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

MINERIA DE DATOS

“ESTUDIO PREDICTIVO SOBRE LAS ELECCIONES PRESIDENCIALES DEL AÑO 2024 – MANUAL TECNICO”

Contenido

Introducción 4
Prerrequisitos 5
Herramientas y Librerías Necesarias 5
1. Instalación de Visual Studio Code (VS Code) 6
1.1.- Descarga 6
1.3.- instalación 6
1.5.- ruta 7
1.7.- Tareas 8
1.9.- Confirmación 9
2.- Configuración Inicial 10
2.2.-Extenciones 10
2.3.- librerías 11
2.4.- Python 11
3.- recolección de los datos 14
3.2.- Registro 14
3.3.- búsqueda de datos 14
3.8.- Descarga de nuestros datos: 16
4.- Ejecución de los códigos 17
4.1.- Separador de comentarios 17
4.2.-Tokenizador 19
4.3.-Diccionario 20
4.4-Entrenamiento 22
4.5.- regresión lineal 30
Conclusiones 36
Referencias 37

Introducción

El presente manual técnico proporciona una guía detallada para realizar un análisis predictivo de opiniones en la red social YouTube, específicamente en el contexto de las elecciones presidenciales de 2024 en México. El objetivo principal de este análisis es predecir la tendencia de opinión de los usuarios hacia las candidatas Claudia Sheinbaum Pardo por la coalición “Juntos Haremos Historia” y Xóchitl Gálvez Ruiz por la coalición “Fuerza y Corazón por México”, con el fin de anticipar quién podría ocupar el mando presidencial.

A lo largo de este manual, se abordarán diferentes aspectos técnicos, desde la configuración del entorno de desarrollo hasta la implementación de algoritmos de análisis de sentimientos. Se utilizarán herramientas como Visual Studio Code y librerías de Python como pandas, numpy, scikit-learn y nltk para llevar a cabo el análisis de manera eficiente y precisa.

El manual está estructurado de manera que se pueda seguir paso a paso, desde la recolección de datos en YouTube hasta la visualización de tendencias y predicciones. Cada sección incluirá explicaciones detalladas y ejemplos de código para facilitar la comprensión y aplicación práctica de los conceptos presentados.

Al finalizar este manual, se espera que el lector haya adquirido los conocimientos necesarios para realizar análisis predictivos similares en otros contextos, así como una comprensión más profunda del potencial de la minería de datos y el análisis de sentimientos en la toma de decisiones basadas en datos en el ámbito político y social.

Prerrequisitos

- Conocimientos básicos de programación en Python.
- Familiaridad con conceptos de minería de datos y análisis de sentimientos.
- Cuenta en Botster para la recolección de datos.

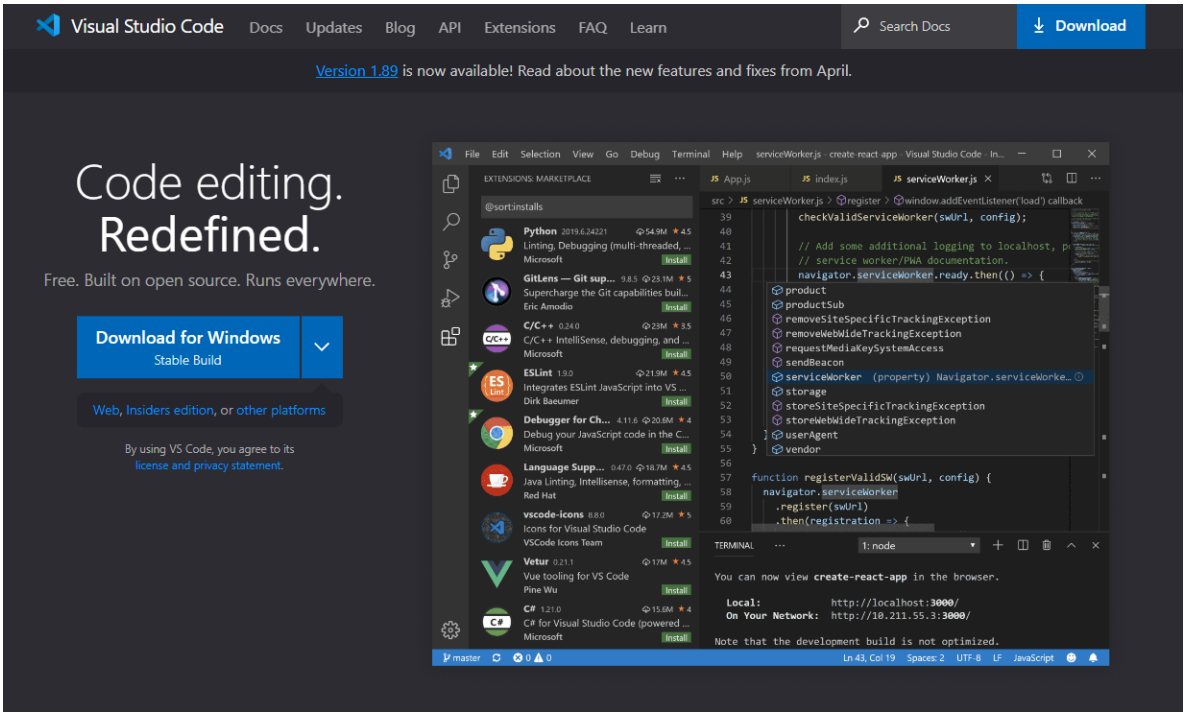
Herramientas y Librerías Necesarias

- Visual Studio Code (VS Code)
- Python 3.9 o superior
- Librerías Python:
- Pandas
- Numpy
- Matplotlib
- Unidecode
- Spacy
- Sklearn

1. Instalación de Visual Studio Code (VS Code)

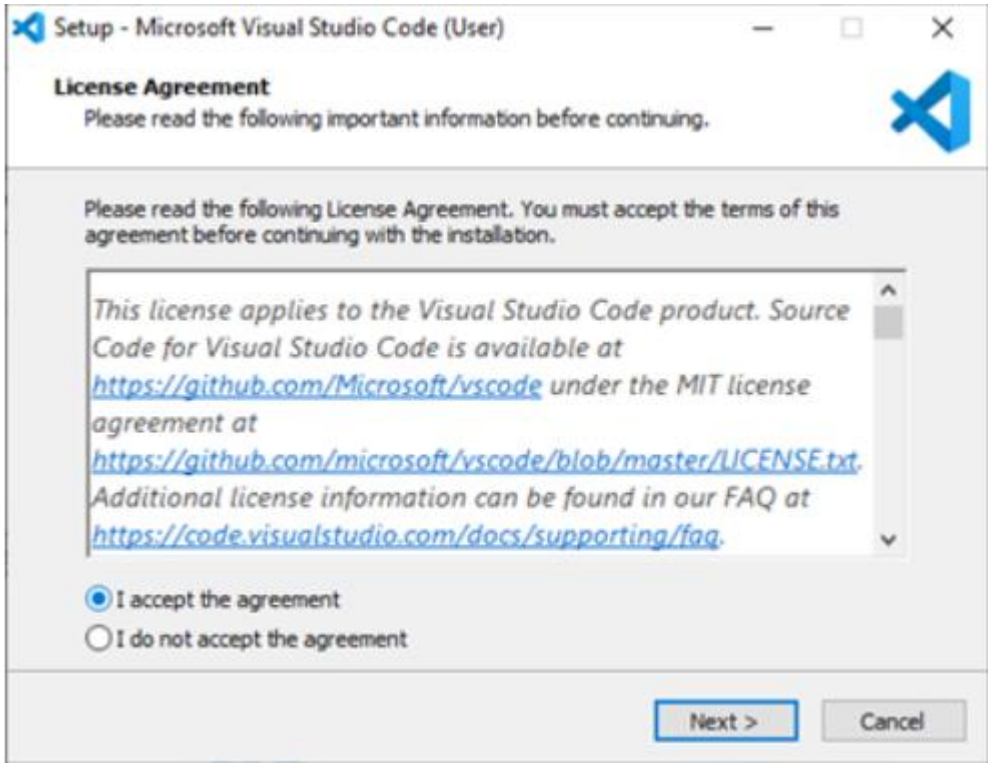
1.1.- Descarga: Descarga e Instalación de VS Code:

1.2.- Visita la página oficial de Visual Studio Code.

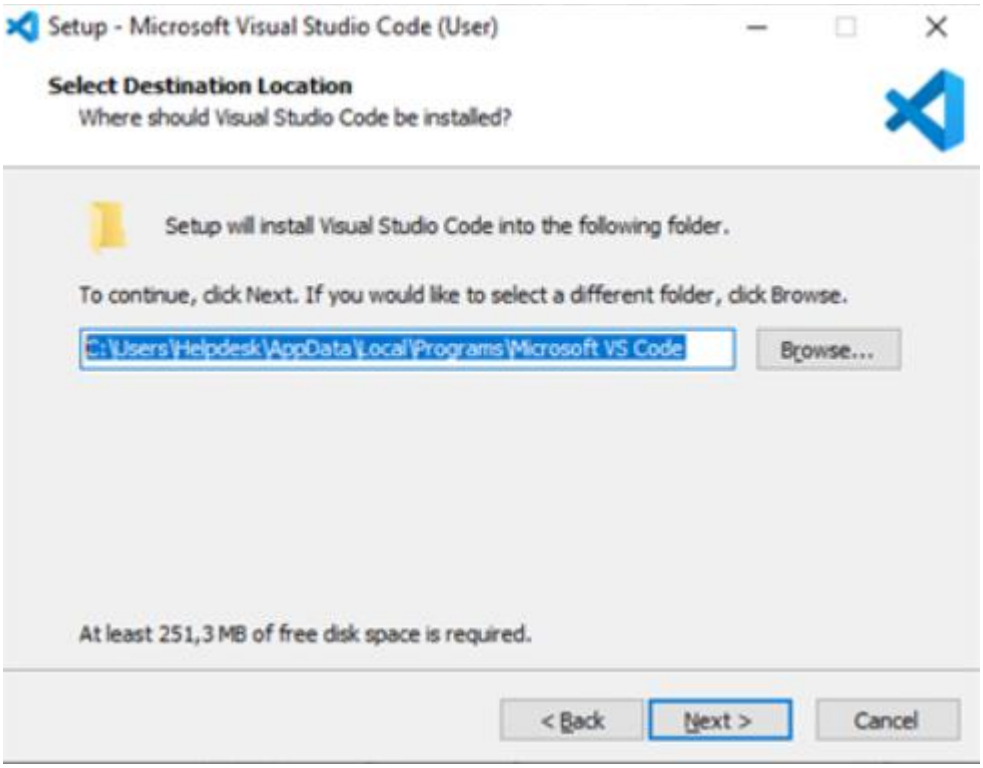


1.3.- instalación: Descarga el instalador correspondiente a tu sistema operativo (Windows, macOS, Linux).

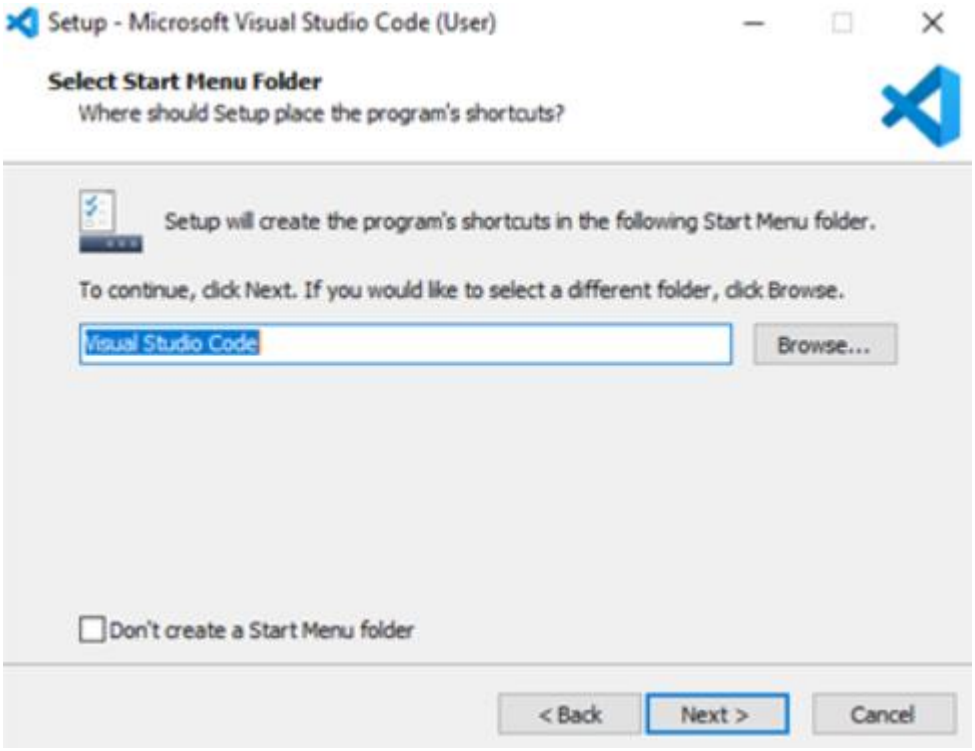
1.4.- Ejecuta el instalador y sigue las instrucciones para completar la instalación.



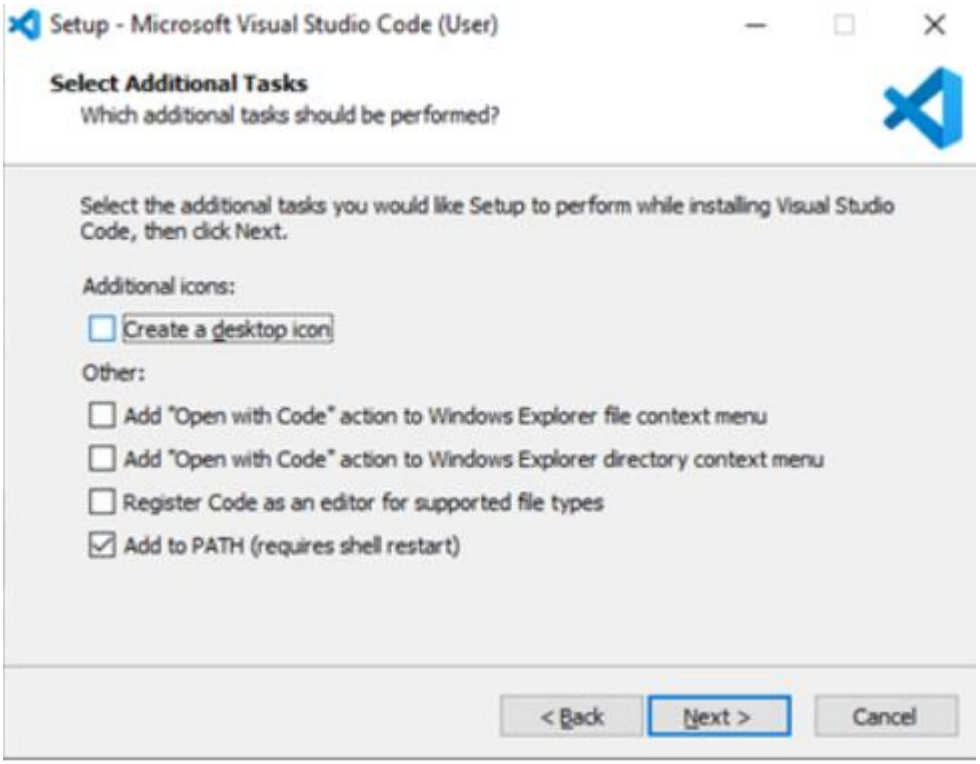
1.5.- ruta: Puedes cambiar la ubicación de la carpeta de instalación o mantener la configuración predeterminada. Haz clic en Next para continuar.



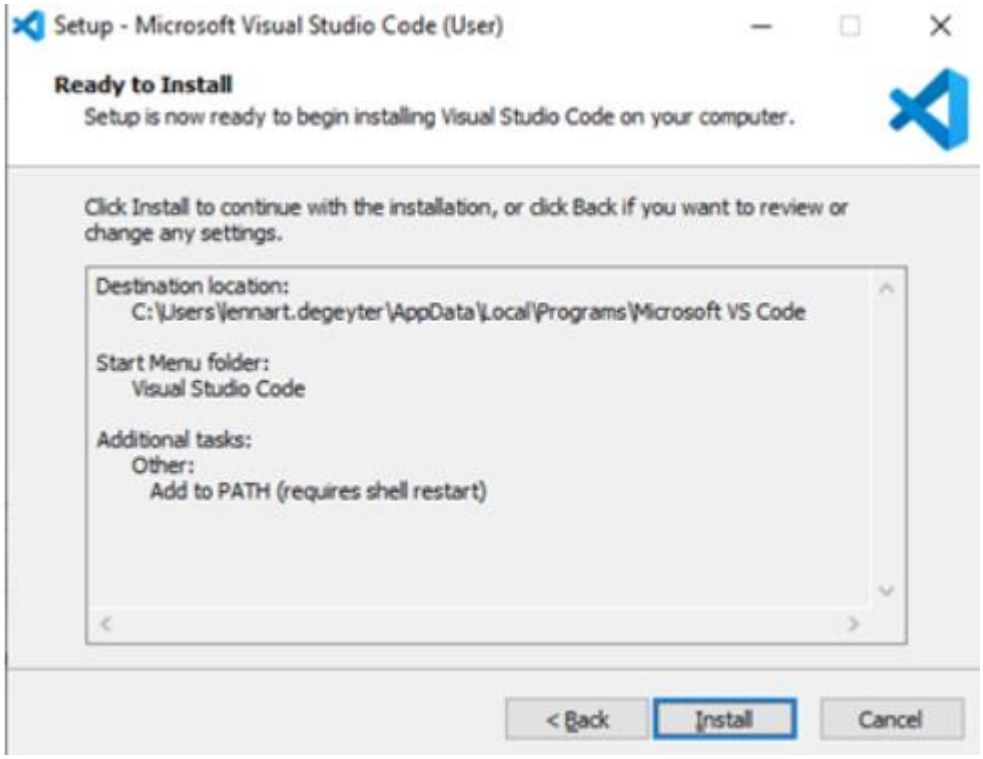
1.6.- Elige si deseas cambiar el nombre de la carpeta de accesos directos en el menú Inicio o si no deseas instalar accesos directos en absoluto. Haz clic en Next.



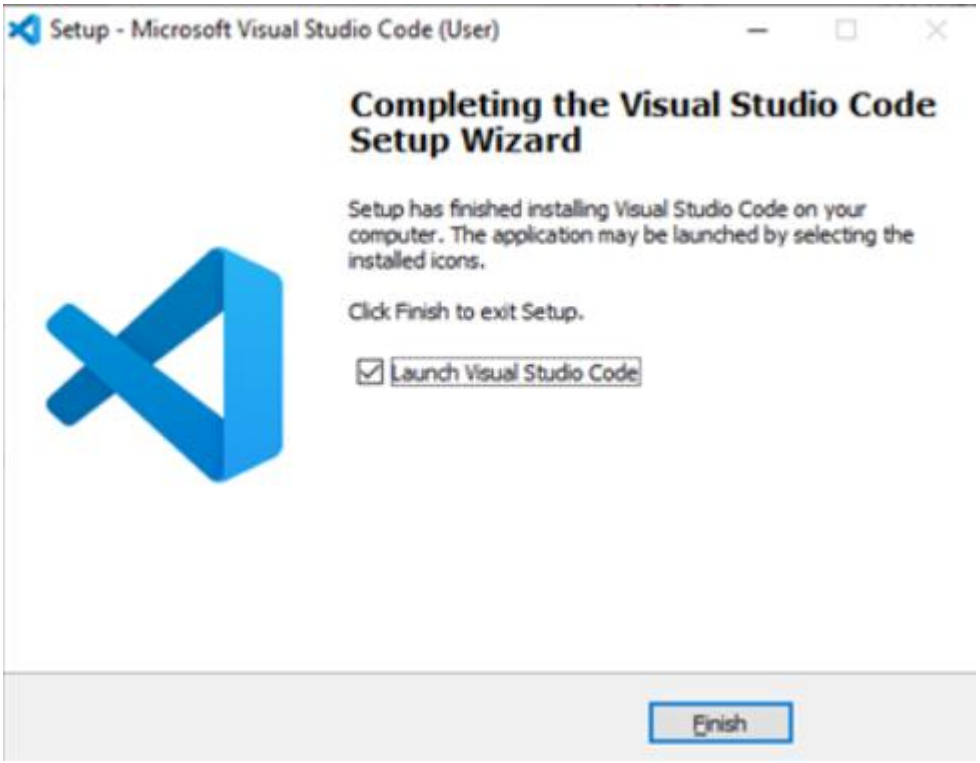
1.7.- Tareas: Selecciona las tareas adicionales, por ej. crear un icono en el escritorio o añadir opciones al menú contextual de Windows Explorer. Haz clic en Next



1.8.- Haz clic en Install para iniciar la instalación.

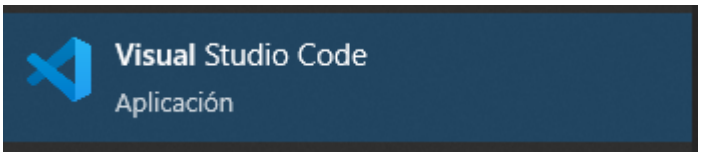


1.9.- Confirmación: Listo



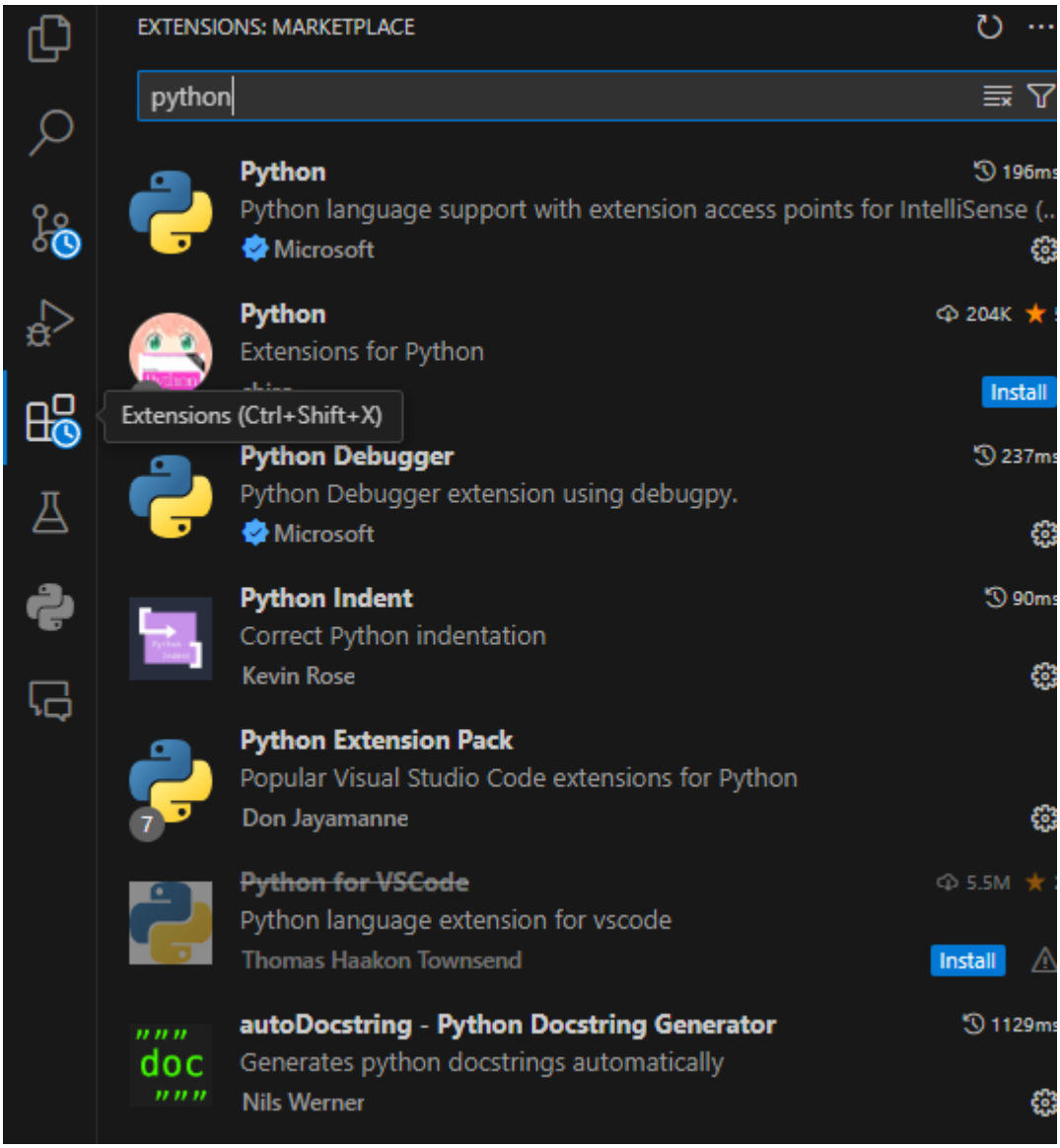
2.- Configuración Inicial

2.1.- Abre Visual Studio Code.



2.2.-Extenciones: Instala las extensiones recomendadas para Python:

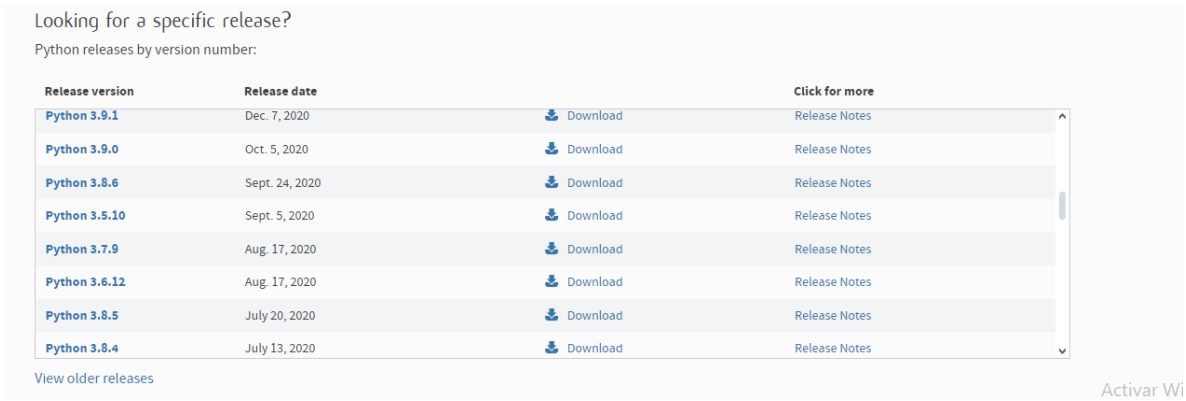
Ve a la sección de Extensiones (icono de cuadritos en la barra lateral izquierda o Ctrl+Shift+X). Busca e instala las extensiones Python.

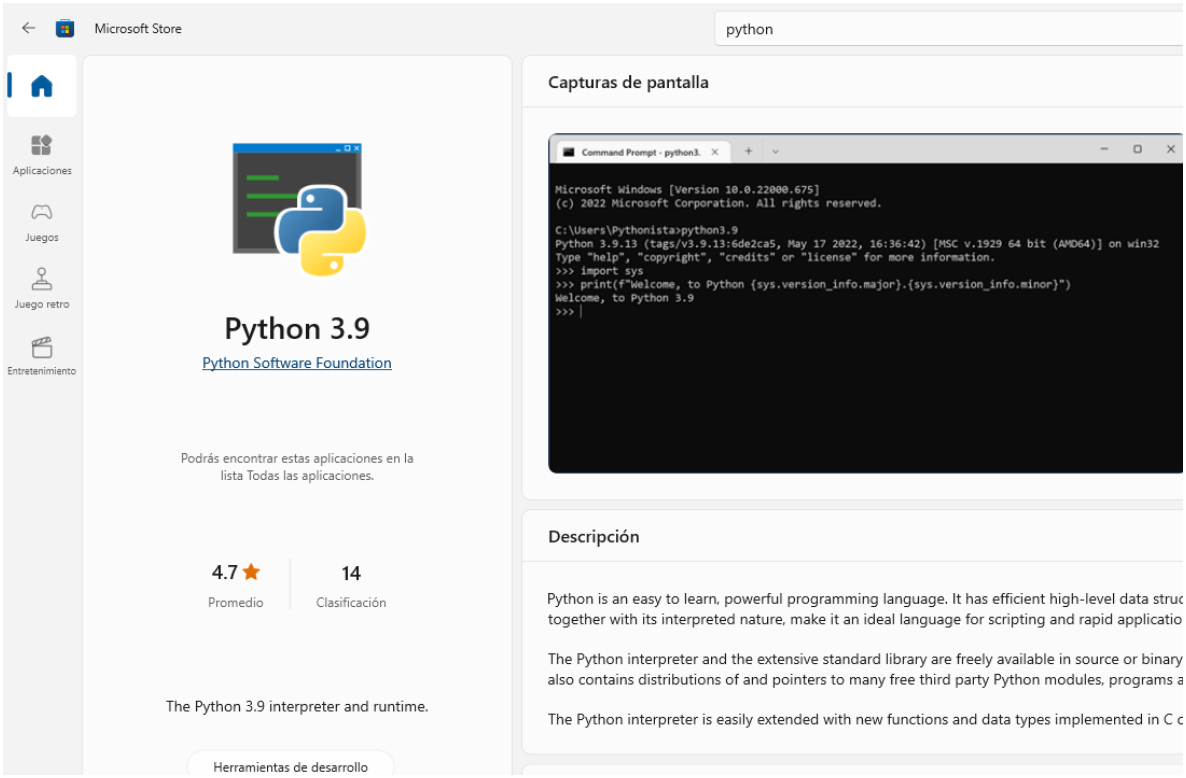


2.3.- librerías: Instalación de Librerías Necesarias

Instalar Python:

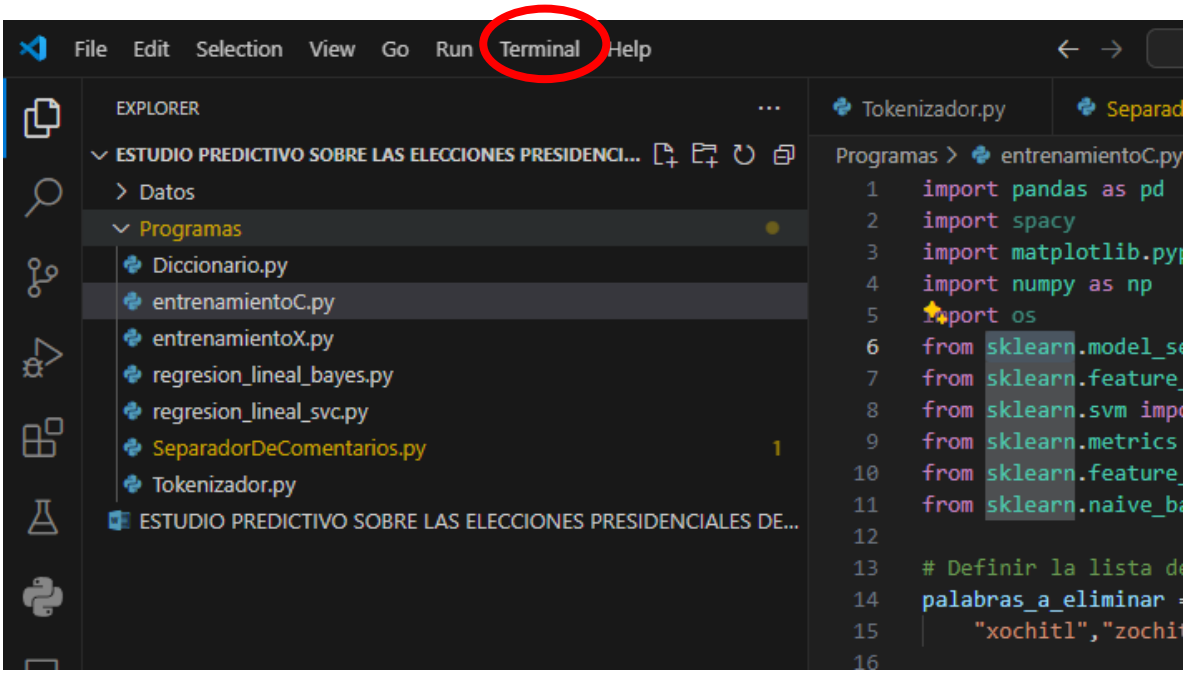
2.4.- Python: Descarga e instala la última versión de Python desde python.org. Asegúrate de agregar Python al PATH durante la instalación. Si estas en Windows lo puedes descargar directamente de la tienda de Windows.



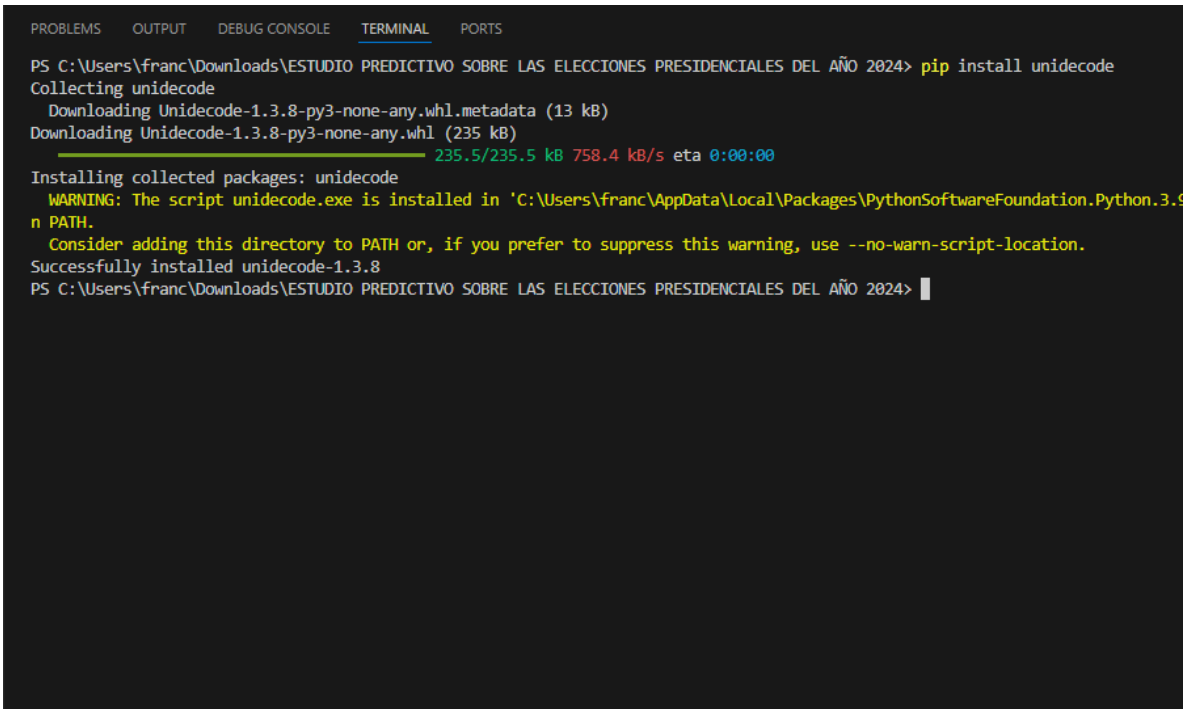


2.5.- Instalamos las bibliotecas necesarias para que nuestros códigos ejecuten perfectamente.

2.6.- Para esto, abrimos nueva terminal en visual studio code.



2.7.- Para descargar las librerías necesarias lo hacemos con el comando: pip Install "librería". Las librerías necesarias para descargar son: pandas, numpy, matplotlib, Unidecode, Spacy y Sklearn.



2.8.-En caso de que te salga algún tipo de error, puedes hacer lo siguiente:

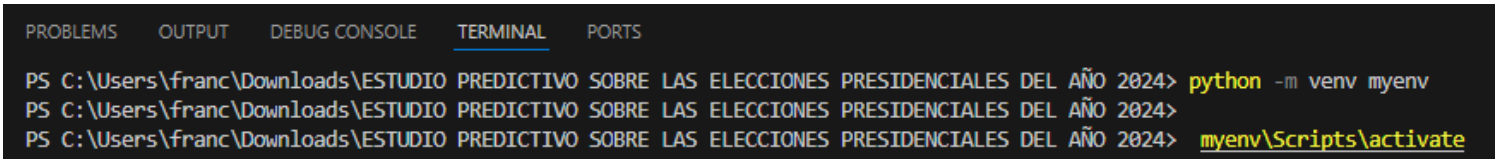
Para acceder abre un entorno virtual para esto desde visual da click en el traingulito que esta hacia abajo y abre command prompt escriban en la terminal generada.

python -m venv myenv

myenv\Scripts\activate

Instalen estas librerias

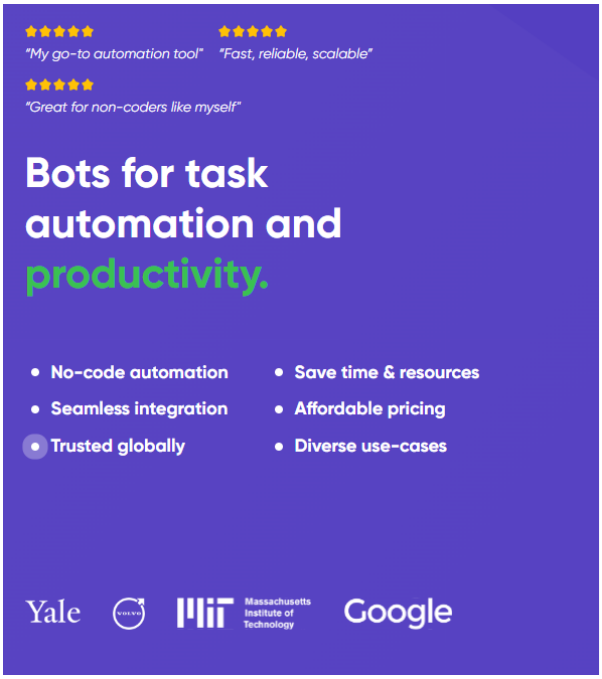
- pip install pandas
- pip install openpyxl
- pip install unidecode
- pip install textblob

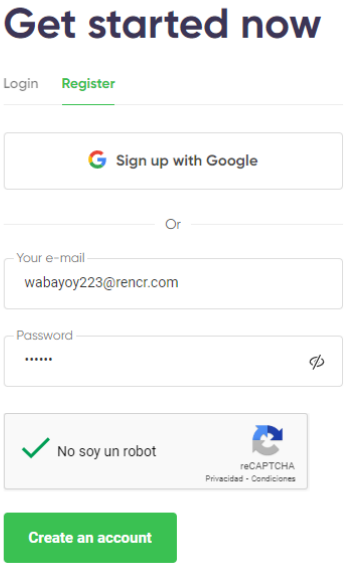


3.- recolección de los datos

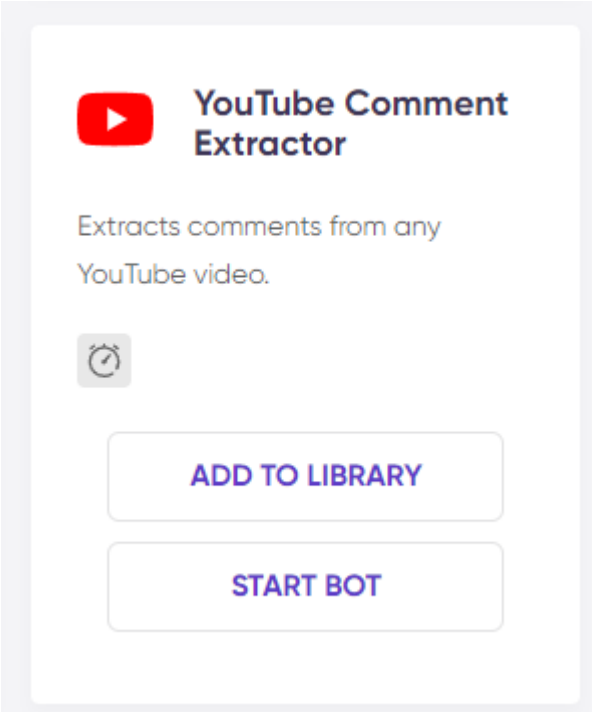
3.1.- Uso de Botster para Extraer Comentarios de YouTube:

3.2.- Registro: Regístrate y configura Botster para extraer comentarios.

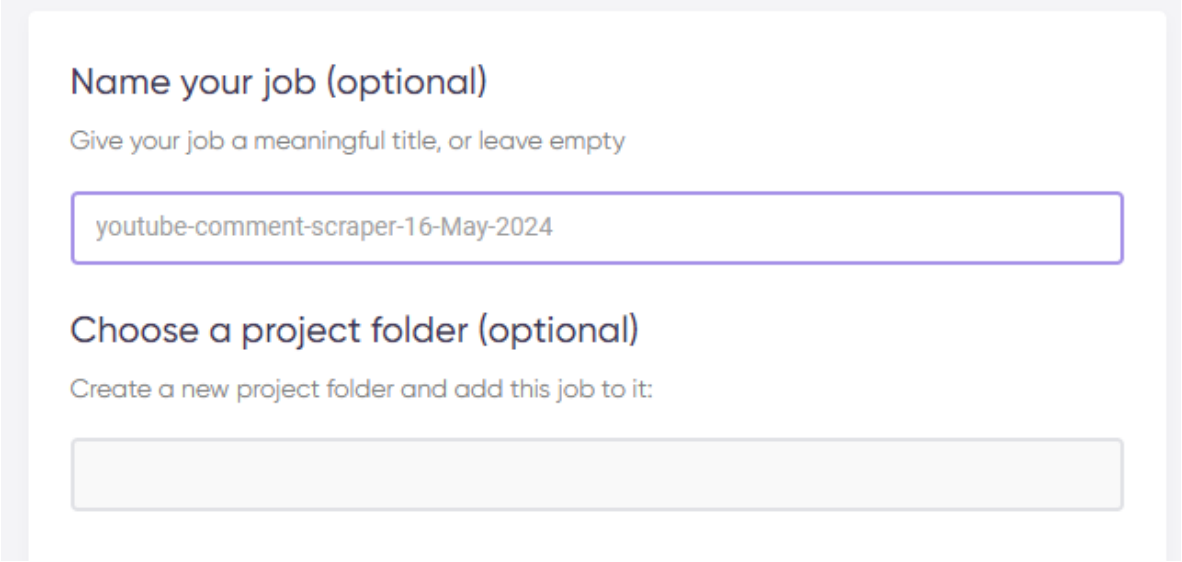




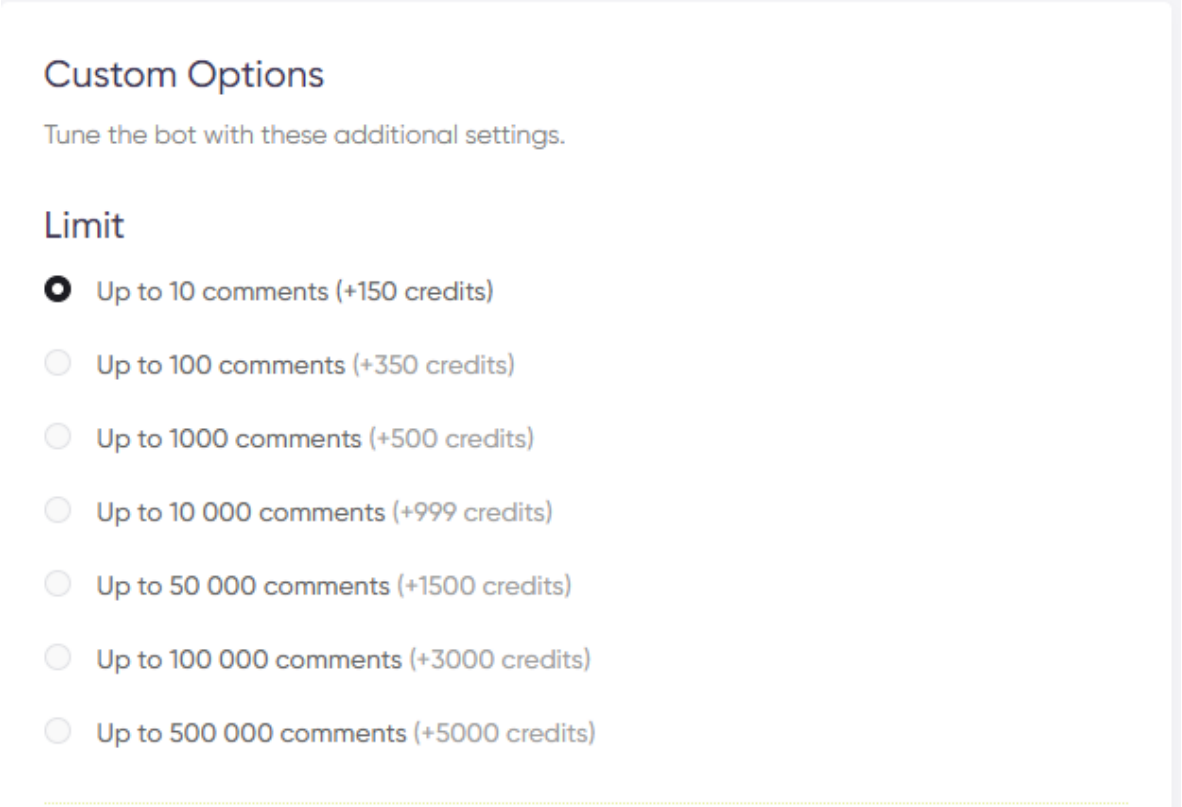
3.3.- búsqueda de datos: Busca YouTube y fíjate en el extractor de comentarios:



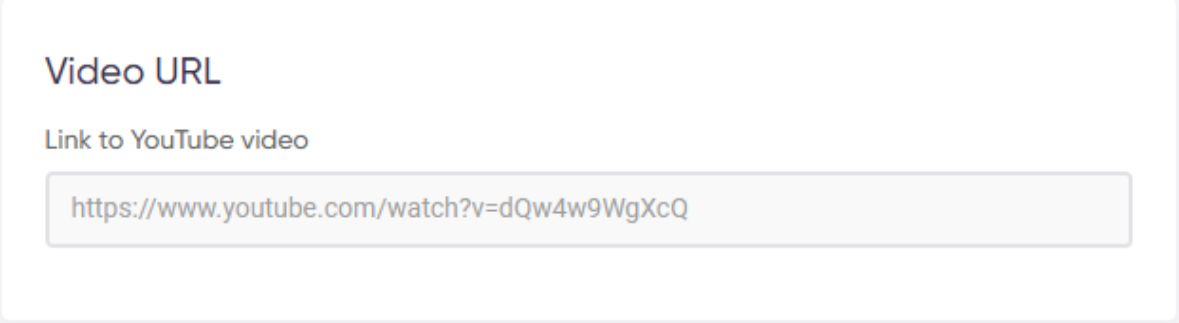
3.4.- Ponle un nombre a tu archivo.



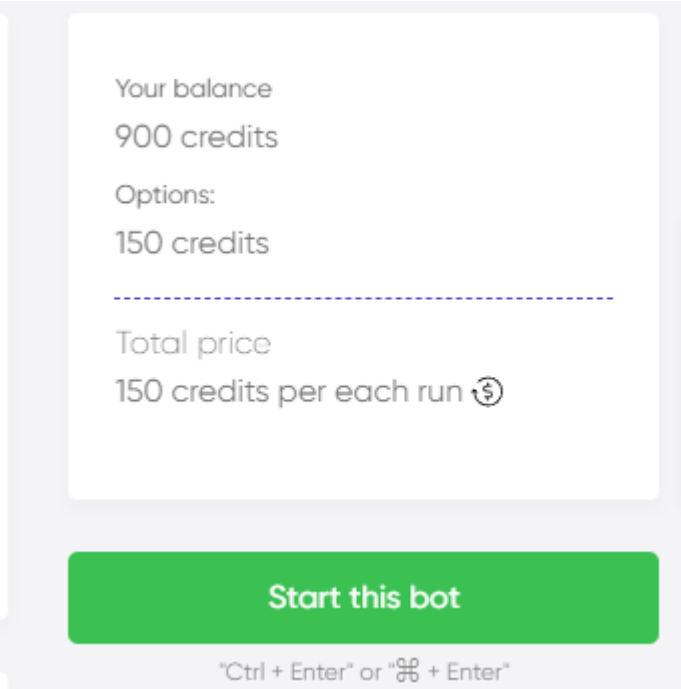
3.5.- Ve los datos que necesites, si puedes pagar para sacar más datos mucho mejor.



3.6.- Pega el URL del video.



3.7.- Inicia el bot.



3.8.- Descarga de nuestros datos: Al final te saldrá una nueva ventana y te pedirá el formato de como descargar tus datos.

DATOS PRUEBA

Created: less than a minute ago Bot: YouTube Comment Extractor

	Video URL	Nickname	User URL	Date
1	https://www.youtube.com/watch?v=xJSIQy33bN4	@fabriziobalderas6299	http://www.youtube.com/@fabriziobalderas6299	2023-05-10T17:39:48.000+00:00
2	https://www.youtube.com/watch?v=xJSIQy33bN4	@axelgengar510	http://www.youtube.com/@axelgengar510	2022-01-22T05:21:53.000+00:00
3	https://www.youtube.com/watch?v=xJSIQy33bN4	@oscarulises1317	http://www.youtube.com/@oscarulises1317	2022-04-05T02:53:10.000+00:00
4	https://www.youtube.com/watch?v=xJSIQy33bN4	@4ngryB0i	http://www.youtube.com/@4ngryB0i	2022-03-03T20:28:02.000+00:00
5	https://www.youtube.com/watch?v=xJSIQy33bN4	@kevinquezada2293	http://www.youtube.com/@kevinquezada2293	2022-01-29T13:09:03.000+00:00
6	https://www.youtube.com/watch?v=xJSIQy33bN4	@ReyesVF_	http://www.youtube.com/@ReyesVF_	2022-01-22T18:38:20.000+00:00
7	https://www.youtube.com/watch?v=xJSIQy33bN4	@raymundo7658	http://www.youtube.com/@raymundo7658	2022-01-18T03:22:23.000+00:00
8	https://www.youtube.com/watch?v=xJSIQy33bN4	@walterortiz4988	http://www.youtube.com/@walterortiz4988	2022-11-10T07:00:04.000+00:00
9	https://www.youtube.com/watch?v=xJSIQy33bN4	@gustavosandovalcastro2369	http://www.youtube.com/@gustavosandovalcastro2369	2022-01-29T02:05:46.000+00:00
10	https://www.youtube.com/watch?v=xJSIQy33bN4	@cristianramirez9699	http://www.youtube.com/@cristianramirez9699	2022-12-03T03:12:07.000+00:00

4.- Ejecución de los códigos

Una vez hecho todo lo anterior, es momento de ejecutar nuestros códigos.

4.1.- Separador de comentarios: Primero comenzaremos con el separador de comentarios. Este código lee un archivo Excel con comentarios, identifica si mencionan a Xóchitl Gálvez o Claudia Sheinbaum utilizando listas de palabras clave, y clasifica los comentarios en dos diccionarios según la mención. Luego guarda los resultados en dos archivos Excel separados, uno para cada persona mencionada.

```
import pandas as pd
from unidecode import unidecode
import os
#-------------------------------------------------------------------------------
----------------------------------------
#-------------- IDENTIFICAMOS QUE COMENTARIOS NOS SIRVEN PARA PODER HACER EL
ANALISIS -------------------------------
#-------------------------------------------------------------------------------
----------------------------------------
def verificar_palabras_clave(texto, lista1, lista2):
    palabras_xo = [palabra.lower() for palabra in lista1]
    palabras_cl = [palabra.lower() for palabra in lista2]

    encontrado_xo = any(palabra in texto for palabra in palabras_xo)
    encontrado_cl = any(palabra in texto for palabra in palabras_cl)

    if encontrado_xo and encontrado_cl:
        return "AMBOS"
    elif encontrado_xo:
        return "xochitl"
    elif encontrado_cl:
        return "claudia"
    else:
        return None
#-------------------------------------------------------------------------------
-----------
#------------------------------- PARTE PRINCIPAL --------------------------
------------
#-------------------------------------------------------------------------------
-----------

#Lectura de nuestro archivo
ruta = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\comentarios_iniciales.xlsx"
df = pd.read_excel(ruta)
```

```
# Crear un diccionario para almacenar los datos
datos = {}
for index, fila in df.iterrows():
    # Crear una lista para almacenar los valores de esta fila
    valores = [fila[columna] for columna in df.columns]
    # Asignar la lista de valores al identificador correspondiente en el
diccionario
    datos[index] = valores


#Listas auxiliares para poder determianr a quien va dirigido el comentario
xo = ["xochitl","zochitl", "xg", "prian","sochitl",
"cochil","galvez","gelatina","gelatinas","debate","debates","botarga","xochi
tlovers","xochitlover","pri","pan","alito"]
cl = ["claudia","sheinbaum ",
"shenbaun","4t","obrador","amlo","morena","plan
c","cheinbaun","transformacion","4ta","2do piso","momia","hielo"]

#Diccionarios para almacenar comentario de xochitl y claudia
dic_clau={}
dic_xo={}
x=0 ; c=0

for i in range(len(datos)):
    texto = datos[i][2]
    resultado = verificar_palabras_clave(texto, xo, cl)
    if resultado:
        if resultado == "claudia":
            dic_clau[c] = [datos[i][0],datos[i][1],texto]
            c += 1
        elif resultado == "xochitl":
            dic_xo[x] = [datos[i][0],datos[i][1],texto]
            x += 1
        else:
            # Separar el texto en dos partes usando la palabra clave
encontrada
            palabra_clave = [palabra for palabra in xo + cl if palabra in
texto][0]
            indice_palabra = texto.index(palabra_clave)
            partes_texto = texto.split(palabra_clave, 1)
            c1 = partes_texto[0].strip() + " " + palabra_clave  # Primera
parte del texto
            c2 = partes_texto[1].strip()  # Segunda parte del
texto
```

```
            # Verificar cada parte para determinar si pertenece a xochitl o
claudia
            for comentario, indice in ((c1, c), (c2, x)):
                resultado = verificar_palabras_clave(comentario, xo, cl)
                if resultado == "claudia":
                    dic_clau[indice] = [datos[i][0],datos[i][1],comentario]
                    c += 1
                elif resultado == "xochitl":
                    dic_xo[indice] = [datos[i][0],datos[i][1],comentario]
                    x += 1


#Pasar los diccionarios generados a un excel para Claudia
df = pd.DataFrame(dic_clau.values(), index=dic_clau.keys(),
columns=['Usuario','Fecha', 'Texto'])
nombre_archivo = 'Claudia_iniciales.xlsx'
df.to_excel(nombre_archivo, index_label='ID')
print(f'Se ha guardado el diccionario en el archivo "{nombre_archivo}"')

#Pasar los diccionarios generados a un excel para Xochitl
df = pd.DataFrame(dic_xo.values(), index=dic_xo.keys(),
columns=['Usuario','Fecha', 'Texto'])
nombre_archivo = 'Xochitl_iniciales.xlsx'
df.to_excel(nombre_archivo, index_label='ID')
print(f'Se ha guardado el diccionario en el archivo "{nombre_archivo}"')
```

4.2.-Tokenizador: De ahí, seguiremos con el código, llamado “tokenizador”. Este código lee archivos Excel con comentarios, tokeniza y lematiza los textos utilizando SpaCy para obtener una versión procesada de los comentarios, y guarda los resultados en nuevos archivos Excel.

```
import os
import pandas as pd
import spacy

def leer_archivo(ruta):
    df = pd.read_excel(ruta)
    datos = df['Texto']
    fecha = df['Fecha']
    return datos,fecha

def tokenizar(datos):
    nlp = spacy.load('es_core_news_md')  # Indicamos el idioma (español)
```

```
    tokens_filtrados=[]
    for texto in datos:
        texto_tokens = nlp(texto)  # Convertir a tokens
        tokens = [token.lemma_ for token in texto_tokens if not
(token.is_space or token.is_punct or token.like_num or (len(token.text) <=
3) or token.like_url or token.is_punct)]
        tokens_filtrados.append(' '.join(tokens))  # Unir tokens filtrados
    return tokens_filtrados

def guardar_excel(nombre_archivo,fecha,datos,tokens):
    resultados = pd.DataFrame({'Fecha': fecha,'Comentario':
datos,'Tokens':tokens})
    resultados.to_excel(nombre_archivo,index=False)
    print("El archivo "+nombre_archivo +" se guardo correctamente")


#Guardar los datos de claudia
datos,fecha = leer_archivo(r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\Claudia_iniciales.xlsx")
tokens = tokenizar(datos)
guardar_excel("Claudia_Tokens.xlsx",fecha,datos,tokens)

#Guardar los datos de xochitl
datos,fecha = leer_archivo(r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\Xochitl_iniciales.xlsx")
tokens = tokenizar(datos)
guardar_excel("Xochitl_Tokens.xlsx",fecha,datos,tokens)
```

4.3.-Diccionario: Luego, nos pasamos al código “diccionario”. El código carga un archivo de Excel que contiene comentarios (columna 'Tokens') y asigna a cada comentario una etiqueta de sentimiento ('positivo', 'negativo' o 'neutro') basada en un diccionario de palabras asociadas a sentimientos positivos y negativos. Luego, guarda los comentarios con sus etiquetas de sentimiento en un nuevo archivo Excel llamado "train_data.xlsx".

```
import pandas as pd
import os

# Función para obtener etiqueta de sentimiento
def obtener_etiqueta_sentimiento(token):
    if isinstance(token, float) and pd.isna(token):  # Verificar si el valor
es NaN
        return "neutro"
    positivos = 0
    negativos = 0
    for palabra in token.split():
```

```
        if palabra in palabras_sentimiento:
            if palabras_sentimiento[palabra] == "positivo":
                positivos += 1
            elif palabras_sentimiento[palabra] == "negativo":
                negativos += 1

    if positivos > negativos:
        return "positivo"
    elif negativos > positivos:
        return "negativo"
    else:
        return "neutro"

# Cargar datos y definir diccionario de sentimientos
palabras_sentimiento = {
    #Palabras Positivas
    "apoyar": "positivo", "presidenta": "positivo", "ir": "positivo",
"abogar": "positivo", "alinear": "positivo", "bravo": "positivo",
    "presidente": "positivo", "excelente": "positivo", "progresar":
"positivo", "votar": "positivo", "avalar": "positivo",
    "felicitar": "positivo", "capaz": "positivo", "honestidad":
"positivo","calidad": "positivo", "elegir": "positivo", "arriba":
"positivo",
    "ganar": "positivo", "mucho": "positivo","respaldar": "positivo",
"felicidad": "positivo", "alegria": "positivo", "mejor": "positivo",
    "favorita": "positivo", "bien": "positivo", "ganar": "positivo",
"felicidad": "positivo", "favorecer": "positivo", "efectivo": "positivo",
    "respeto": "positivo", "admirar": "positivo", "confiar": "positivo",
"confianza": "positivo", "amar": "positivo", "primera": "positivo",
    "celebrar": "positivo","valorar": "positivo", "elogiar": "positivo",
"inspirar": "positivo", "impulsar": "positivo","motivar": "positivo",
    "enriquecer": "positivo","empoderar": "positivo","promover":
"positivo","libertad": "positivo", "democracia": "positivo",
    "increible": "positivo","genial": "positivo","fabuloso":
"positivo","notable": "positivo","estupendo": "positivo", "seguir":
"positivo",
    "continuar": "positivo","transformar": "positivo","sobresalir":
"positivo","crear": "positivo","confirmar": "positivo",
    "construir": "positivo","seguridad": "positivo","adelante":
"positivo","victoria": "positivo","viva": "positivo","garantizar":
"positivo",
    "ayudar": "positivo","estar": "positivo","dedicar":
"positivo","chingon": "positivo","fregon": "positivo","corazon": "positivo",
    #Palabras negativas
```

```
    "talones": "negativo", "perder": "negativo", "pendejo": "negativo",
"pendeja": "negativo", "terror": "negativo", "corrupcion":"negativo",
    "desesperar": "negativo", "fallar": "negativo", "reir": "negativo",
"loco": "negativo","idiota": "negativo", "condenar": "negativo",
    "rata": "negativo", "robar": "negativo", "vender": "negativo", "pasado":
"negativo", "pobre": "negativo", "rechazar": "negativo",
    "arrastrada": "negativo", "ignorante": "positivo", "verguenza":
"negativo", "bote": "negativo", "carcel": "negativo", "sarcasmo":
"negativo",
    "narco": "negativo", "mentir": "negativo", "perder": "negativo",
"culero": "negativo","sarcasmo": "negativo", "desacreditar": "negativo",
    "incompetente": "negativo","deshonesto": "negativo","engañar":
"negativo","inepto": "negativo","irresponsable": "negativo",
    "desleal": "negativo","tramposo": "negativo","ineficiente":
"negativo","manipulador": "negativo","sucio": "negativo", "ratero":
"negativo",
    "ultima": "negativo","lejos": "negativo","mal": "negativo","peor":
"negativo","desafortunado": "negativo", "hielo": "negativo",
    "botarga": "negativo", "abajo": "negativo", "fuera": "negativo",
"imbecil": "negativo", "tarado": "negativo", "lamentar": "negativo",
    "arrepentir": "negativo", "verdulera": "negativo", "anciana":
"negativo", "vieja": "negativo", "bruja": "negativo", "chencha": "negativo",
    "floja": "negativo", "peligro": "negativo", "dañar": "negativo",
"botarga": "negativo", "muerte": "negativo", "agresion": "negativo",
    "atras": "negativo","abandonar": "negativo","rabia":
"negativo","tristeza": "negativo","depresion": "negativo","derrotar":
"negativo",
    "traicion": "positivo","momia": "positivo", "hielo": "positivo",
}

ruta = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\Datos_Entrenamiento.xlsx"
df = pd.read_excel(ruta)
etiquetas = [obtener_etiqueta_sentimiento(token) for token in df['Tokens']]
# Obtener etiquetas de sentimiento para cada comentario
df['Sentimiento_Predicho'] = etiquetas # Añadir las etiquetas al DataFrame
df.to_excel("train_data.xlsx", index=False)
```

4.4-Entrenamiento: Ahora nos vamos a los códigos de entrenamiento, tanto para claudia y Xóchitl. Este código carga dos conjuntos de datos de comentarios políticos (uno relacionado con Claudia y otro con Xochitl), los preprocesa eliminando palabras específicas y los entrena con modelos de SVM y Naive Bayes para predecir el sentimiento de los comentarios. Luego, realiza predicciones sobre nuevos textos y guarda los resultados,

incluidas las probabilidades de cada sentimiento y las palabras más asociadas según Naive Bayes, en archivos Excel separados ("resultados_Claudia.xlsx" y "resultados_Xochitl.xlsx").

```
import pandas as pd
import spacy
import matplotlib.pyplot as plt
import numpy as np
import os
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.svm import SVC
from sklearn.metrics import classification_report
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Definir la lista de palabras a eliminar
palabras_a_eliminar = ["claudia","sheinbaum ",
"shenbaun","4t","obrador","amlo","morena","plan
c","cheinbaun","transformacion","4ta","2do piso","vato","maynez"
    "xochitl","zochitl", "xg", "prian","sochitl",
"cochil","galvez","gelatina","gelatinas","debate","debates","botarga","xochi
tlovers","xochitlover","pri","pan","alito"]

def eliminar_palabras(texto):
    # Verificar si el valor es un string
    if isinstance(texto, str):
        palabras = texto.split()
        palabras_filtradas = [palabra for palabra in palabras if
palabra.lower() not in palabras_a_eliminar]
        return ' '.join(palabras_filtradas)
    else:
        # Si no es un string, retornar el mismo valor
        return texto
ruta1 = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\train_data.xlsx"
ruta2 = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\Claudia_Tokens.xlsx" #Cargar datos para poder entrenar
df = pd.read_excel(ruta1)
df2 = pd.read_excel(ruta2)

# Eliminar filas con NaN en la columna 'Tokens'
df = df.dropna(subset=['Tokens'])
# Verificar si hay documentos que no sean cadenas de texto después de
eliminar NaN
df2 = df2[df2['Tokens'].apply(lambda x: isinstance(x, str))]
```

```
# Verificar si hay documentos vacíos
df2 = df2[df2['Tokens'].apply(lambda x: len(x.strip()) > 0)]

# Convertir el campo 'Fecha' a formato de fecha especificando dayfirst=True
df2['Fecha'] = pd.to_datetime(df2['Fecha'], dayfirst=True)

# Extraer solo la parte de la fecha y eliminar la parte de la hora
df2['Fecha'] = df2['Fecha'].dt.date

# Ordenar el DataFrame por la columna 'Fecha' de manera descendente
df2 = df2.sort_values(by='Fecha', ascending=True)

df3 = df2.iloc[:, 2].apply(eliminar_palabras)
# Vectorizacion de caracteristicas
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['Tokens'])
y = df['Sentimiento_Predicho']

# Dividir el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Entrenamiento del modelo SVM
svm_model = SVC(kernel='linear')
svm_model.fit(X_train, y_train)

# Evaluación del modelo SVM
y_pred = svm_model.predict(X_test)
print("Resultados del modelo SVM")
print(classification_report(y_test, y_pred, zero_division=1))

# Predicción de sentimientos para nuevos textos con SVM
new_texts_vectorized = vectorizer.transform(df3)
predictions = svm_model.predict(new_texts_vectorized)

# Entrenamiento y predicciones con el modelo Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
predictions_nb = nb_model.predict(new_texts_vectorized)
proba_predictions_nb = nb_model.predict_proba(new_texts_vectorized) #Prob de
que pertenezca a su clase
# Imprimir la forma de proba_predictions_nb
print(proba_predictions_nb.shape)
```

```
# Calculando las probabilidades para cada clase
probabilities = nb_model.predict_proba(new_texts_vectorized)

# Probabilidad de ser positivo
prob_pos = probabilities[:, 1]  # Tomando la columna de probabilidad de
clase positiva

# Probabilidad de ser negativo
prob_neg = probabilities[:, 0]  # Tomando la columna de probabilidad de
clase negativa

# Probabilidad de ser neutro (si es necesario)
prob_neutral = 1 - (prob_pos + prob_neg)

if len(prob_neutral) != len(df2):
    print("Error: La longitud de prob_neutral no coincide con la longitud de
df2")
    exit()
else:
    print("Todo bien")
# Crear un DataFrame para almacenar los resultados
results_df = pd.DataFrame()

# Copiar las columnas 'Fecha' y 'Comentario' de df2 al nuevo DataFrame
results_df['Fecha'] = df2['Fecha']
results_df['Comentario'] = df2['Comentario']
results_df['Token_filtrado'] = df2['Tokens']

# Agregar las predicciones y probabilidades de ambos modelos al DataFrame
results_df['Sentimiento_Predicho_SVC'] = predictions
results_df['Sentimiento_por_NaiveBayes'] = predictions_nb
results_df['Probabilidad'] = np.max(proba_predictions_nb, axis=1)

# Reiniciar los índices de results_df
results_df.reset_index(drop=True, inplace=True)

# Crear una lista para almacenar las probabilidades calculadas
probabilities_nb = []

# Iterar a través de las filas del DataFrame
for index, row in results_df.iterrows():
    sentiment = row['Sentimiento_por_NaiveBayes']

    if sentiment == 'Positivo':
```

```
        probabilities_nb.append(proba_predictions_nb[index, 1])  #
Probabilidad de ser positivo
    elif sentiment == 'Negativo':
        probabilities_nb.append(proba_predictions_nb[index, 0])  #
Probabilidad de ser negativo
    else:
        probabilities_nb.append(proba_predictions_nb[index, 2])  #
Probabilidad de ser neutro

# Agregar la lista de probabilidades al DataFrame como una nueva columna
results_df['Probabilidad_por_NaiveBayes'] = probabilities_nb

# Obtener las palabras asociadas por Naive Bayes
palabras_asociadas_nb = []

# Convertir la representación vectorizada de los nuevos textos de vuelta a
texto
new_texts = vectorizer.inverse_transform(new_texts_vectorized)

# Iterar a través de las filas del DataFrame
for index, _ in results_df.iterrows():
    # Obtener las palabras asociadas al comentario
    palabras_asociadas = new_texts[index]

    # Si hay palabras asociadas, tomar la primera (asumimos que solo hay
una)
    if len(palabras_asociadas) > 0:
        palabra_asociada = palabras_asociadas[0]
    else:
        palabra_asociada = ""

    # Agregar la palabra asociada a la lista
    palabras_asociadas_nb.append(palabra_asociada)

# Agregar la lista de palabras asociadas al DataFrame como una nueva columna
results_df['Palabra_Asociada_por_NaiveBayes'] = palabras_asociadas_nb


# Guardar el DataFrame en un archivo Excel
results_df.to_excel("resultados_Claudia.xlsx", index=False)

print("Resultados guardados en resultados_Claudia.xlsx")
```

```
import pandas as pd
import spacy
import matplotlib.pyplot as plt
import numpy as np
import os
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.svm import SVC
from sklearn.metrics import classification_report
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Definir la lista de palabras a eliminar
palabras_a_eliminar = ["claudia","sheinbaum ",
"shenbaun","4t","obrador","amlo","morena","plan
c","cheinbaun","transformacion","4ta","2do piso","vato","maynez"
    "xochitl","zochitl", "xg", "prian","sochitl",
"cochil","galvez","gelatina","gelatinas","debate","debates","botarga","xochi
tlovers","xochitlover","pri","pan","alito"]

def eliminar_palabras(texto):
    # Verificar si el valor es un string
    if isinstance(texto, str):
        palabras = texto.split()
        palabras_filtradas = [palabra for palabra in palabras if
palabra.lower() not in palabras_a_eliminar]
        return ' '.join(palabras_filtradas)
    else:
        # Si no es un string, retornar el mismo valor
        return texto
ruta1 = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\train_data.xlsx"
ruta2 = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\Xochitl_Tokens.xlsx" #Cargar datos para poder entrenar
df = pd.read_excel(ruta1)
df2 = pd.read_excel(ruta2)

# Eliminar filas con NaN en la columna 'Tokens'
df = df.dropna(subset=['Tokens'])
# Verificar si hay documentos que no sean cadenas de texto después de
eliminar NaN
df2 = df2[df2['Tokens'].apply(lambda x: isinstance(x, str))]

# Verificar si hay documentos vacíos
df2 = df2[df2['Tokens'].apply(lambda x: len(x.strip()) > 0)]
```

```
# Convertir el campo 'Fecha' a formato de fecha especificando dayfirst=True
df2['Fecha'] = pd.to_datetime(df2['Fecha'], dayfirst=True)

# Extraer solo la parte de la fecha y eliminar la parte de la hora
df2['Fecha'] = df2['Fecha'].dt.date

# Ordenar el DataFrame por la columna 'Fecha' de manera descendente
df2 = df2.sort_values(by='Fecha', ascending=True)

df3 = df2.iloc[:, 2].apply(eliminar_palabras)
# Vectorizacion de caracteristicas
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['Tokens'])
y = df['Sentimiento_Predicho']

# Dividir el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Entrenamiento del modelo SVM
svm_model = SVC(kernel='linear')
svm_model.fit(X_train, y_train)

# Evaluación del modelo SVM
y_pred = svm_model.predict(X_test)
print("Resultados del modelo SVM")
print(classification_report(y_test, y_pred, zero_division=1))

# Predicción de sentimientos para nuevos textos con SVM
new_texts_vectorized = vectorizer.transform(df3)
predictions = svm_model.predict(new_texts_vectorized)

# Entrenamiento y predicciones con el modelo Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
predictions_nb = nb_model.predict(new_texts_vectorized)
proba_predictions_nb = nb_model.predict_proba(new_texts_vectorized) #Prob de
que pertenezca a su clase
# Imprimir la forma de proba_predictions_nb
print(proba_predictions_nb.shape)

# Calculando las probabilidades para cada clase
probabilities = nb_model.predict_proba(new_texts_vectorized)
```

```
# Probabilidad de ser positivo
prob_pos = probabilities[:, 1]  # Tomando la columna de probabilidad de
clase positiva

# Probabilidad de ser negativo
prob_neg = probabilities[:, 0]  # Tomando la columna de probabilidad de
clase negativa

# Probabilidad de ser neutro (si es necesario)
prob_neutral = 1 - (prob_pos + prob_neg)

if len(prob_neutral) != len(df2):
    print("Error: La longitud de prob_neutral no coincide con la longitud de
df2")
    exit()
else:
    print("Todo bien")
# Crear un DataFrame para almacenar los resultados
results_df = pd.DataFrame()

# Copiar las columnas 'Fecha' y 'Comentario' de df2 al nuevo DataFrame
results_df['Fecha'] = df2['Fecha']
results_df['Comentario'] = df2['Comentario']
results_df['Token_filtrado'] = df2['Tokens']

# Agregar las predicciones y probabilidades de ambos modelos al DataFrame
results_df['Sentimiento_Predicho_SVC'] = predictions
results_df['Sentimiento_por_NaiveBayes'] = predictions_nb
results_df['Probabilidad'] = np.max(proba_predictions_nb, axis=1)

# Reiniciar los índices de results_df
results_df.reset_index(drop=True, inplace=True)

# Crear una lista para almacenar las probabilidades calculadas
probabilities_nb = []

# Iterar a través de las filas del DataFrame
for index, row in results_df.iterrows():
    sentiment = row['Sentimiento_por_NaiveBayes']

    if sentiment == 'Positivo':
        probabilities_nb.append(proba_predictions_nb[index, 1])  #
Probabilidad de ser positivo
    elif sentiment == 'Negativo':
```

```
        probabilities_nb.append(proba_predictions_nb[index, 0])  #
Probabilidad de ser negativo
    else:
        probabilities_nb.append(proba_predictions_nb[index, 2])  #
Probabilidad de ser neutro

# Agregar la lista de probabilidades al DataFrame como una nueva columna
results_df['Probabilidad_por_NaiveBayes'] = probabilities_nb

# Obtener las palabras asociadas por Naive Bayes
palabras_asociadas_nb = []

# Convertir la representación vectorizada de los nuevos textos de vuelta a
texto
new_texts = vectorizer.inverse_transform(new_texts_vectorized)

# Iterar a través de las filas del DataFrame
for index, _ in results_df.iterrows():
    # Obtener las palabras asociadas al comentario
    palabras_asociadas = new_texts[index]

    # Si hay palabras asociadas, tomar la primera (asumimos que solo hay
una)
    if len(palabras_asociadas) > 0:
        palabra_asociada = palabras_asociadas[0]
    else:
        palabra_asociada = ""

    # Agregar la palabra asociada a la lista
    palabras_asociadas_nb.append(palabra_asociada)

# Agregar la lista de palabras asociadas al DataFrame como una nueva columna
results_df['Palabra_Asociada_por_NaiveBayes'] = palabras_asociadas_nb


# Guardar el DataFrame en un archivo Excel
results_df.to_excel("resultados_Xochitl.xlsx", index=False)

print("Resultados guardados en resultados_Xochitl.xlsx")
```

4.5.- regresión lineal: Ahora viene lo bueno, aplicar regresión lineal a todo lo anterior. Este código realiza un análisis de sentimiento sobre los comentarios políticos relacionados con Claudia Sheinbaum. Primero, carga los datos de sentimiento previamente procesados y

los divide en sentimientos positivos y negativos. Luego, agrupa los datos por fecha y calcula el promedio de la probabilidad para cada sentimiento en cada fecha.

Después, ajusta modelos de regresión lineal a los datos históricos de sentimiento negativo y positivo para predecir las probabilidades de sentimiento para los próximos 20 días. Finalmente, grafica los datos históricos y las predicciones de probabilidad para cada sentimiento, tanto para los datos históricos como para las predicciones futuras.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Cargar el archivo Excel
rutaC = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\resultados_Claudia.xlsx"
rutaX = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\resultados_Xochitl.xlsx"

df = pd.read_excel(rutaC)

# Convertir la columna de fechas a tipo datetime
df['Fecha'] = pd.to_datetime(df['Fecha'])

# Filtrar los datos por sentimiento negativo y positivo
df_negativo = df[df['Sentimiento_Predicho_SVC'] == 'negativo']
df_positivo = df[df['Sentimiento_Predicho_SVC'] == 'positivo']

# Agrupar por fecha y sumar el sentimiento y promediar la probabilidad
df_negativo_agrupado = df_negativo.groupby('Fecha').agg({'Probabilidad':
'mean'}).reset_index()
df_positivo_agrupado = df_positivo.groupby('Fecha').agg({'Probabilidad':
'mean'}).reset_index()

# Ajustar un modelo de regresión lineal a los datos históricos de
sentimiento negativo
X_negativo = np.arange(len(df_negativo_agrupado)).reshape(-1, 1)
y_negativo = df_negativo_agrupado['Probabilidad'].values
modelo_negativo = LinearRegression()
modelo_negativo.fit(X_negativo, y_negativo)

# Ajustar un modelo de regresión lineal a los datos históricos de
sentimiento positivo
```

```
X_positivo = np.arange(len(df_positivo_agrupado)).reshape(-1, 1)
y_positivo = df_positivo_agrupado['Probabilidad'].values
modelo_positivo = LinearRegression()
modelo_positivo.fit(X_positivo, y_positivo)

# Predecir los próximos 20 días para sentimiento negativo
proximos_dias_negativo =
pd.date_range(start=df_negativo_agrupado['Fecha'].iloc[-1] +
pd.Timedelta(days=1), periods=20, freq='D')
proximos_indices_negativo = np.arange(len(df_negativo_agrupado),
len(df_negativo_agrupado) + 20).reshape(-1, 1)
predicciones_negativo = modelo_negativo.predict(proximos_indices_negativo)
predicciones_df_negativo = pd.DataFrame({'Fecha': proximos_dias_negativo,
'prediccion_probabilidad_negativo': predicciones_negativo})

# Predecir los próximos 20 días para sentimiento positivo
proximos_dias_positivo =
pd.date_range(start=df_positivo_agrupado['Fecha'].iloc[-1] +
pd.Timedelta(days=1), periods=20, freq='D')
proximos_indices_positivo = np.arange(len(df_positivo_agrupado),
len(df_positivo_agrupado) + 20).reshape(-1, 1)
predicciones_positivo = modelo_positivo.predict(proximos_indices_positivo)
predicciones_df_positivo = pd.DataFrame({'Fecha': proximos_dias_positivo,
'prediccion_probabilidad_positivo': predicciones_positivo})

# Graficar los datos históricos y las predicciones para sentimiento negativo
plt.figure(figsize=(10, 6))
plt.plot(df_negativo_agrupado['Fecha'],
df_negativo_agrupado['Probabilidad'], label='Negativo - Datos Históricos',
marker='o', color='red')
plt.plot(predicciones_df_negativo['Fecha'],
predicciones_df_negativo['prediccion_probabilidad_negativo'],
label='Negativo - Predicción', linestyle='--', marker='o', color='darkred')

# Graficar los datos históricos y las predicciones para sentimiento positivo
plt.plot(df_positivo_agrupado['Fecha'],
df_positivo_agrupado['Probabilidad'], label='Positivo - Datos Históricos',
marker='o', color='green')
plt.plot(predicciones_df_positivo['Fecha'],
predicciones_df_positivo['prediccion_probabilidad_positivo'],
label='Positivo - Predicción', linestyle='--', marker='o',
color='darkgreen')

plt.title('Predicción de Probabilidad por Sentimiento de SVC (Claudia
Sheinbaum)')
```

```
#plt.title('Predicción de Probabilidad por Sentimiento de SVC (Xochitl
Gálvez)')
plt.xlabel('Fecha')
plt.ylabel('Probabilidad Promedio')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Este código realiza un análisis de sentimiento sobre los comentarios políticos relacionados con Xochitl Gálvez utilizando el modelo Naive Bayes. Primero, carga los datos de sentimiento previamente procesados y los divide en sentimientos positivos y negativos. Luego, agrupa los datos por fecha y calcula el promedio de la probabilidad para cada sentimiento en cada fecha.

Después, ajusta modelos de regresión lineal a los datos históricos de sentimiento negativo y positivo para predecir las probabilidades de sentimiento para los próximos 20 días. Finalmente, grafica los datos históricos y las predicciones de probabilidad para cada sentimiento, tanto para los datos históricos como para las predicciones futuras.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Cargar el archivo Excel
rutaC = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\resultados_Claudia.xlsx"
rutaX = r"C:\Users\Admin\Desktop\Prediccion Elecciones
2024\Datos\resultados_Xochitl.xlsx"

df = pd.read_excel(rutaX)

# Convertir la columna de fechas a tipo datetime
df['Fecha'] = pd.to_datetime(df['Fecha'])

# Filtrar los datos por sentimiento negativo y positivo
df_negativo = df[df['Sentimiento_por_NaiveBayes'] == 'negativo']
df_positivo = df[df['Sentimiento_por_NaiveBayes'] == 'positivo']

# Agrupar por fecha y sumar el sentimiento y promediar la probabilidad
df_negativo_agrupado =
df_negativo.groupby('Fecha').agg({'Probabilidad_por_NaiveBayes':
'mean'}).reset_index()
```

```
df_positivo_agrupado =
df_positivo.groupby('Fecha').agg({'Probabilidad_por_NaiveBayes':
'mean'}).reset_index()

# Ajustar un modelo de regresión lineal a los datos históricos de
sentimiento negativo
X_negativo = np.arange(len(df_negativo_agrupado)).reshape(-1, 1)
y_negativo = df_negativo_agrupado['Probabilidad_por_NaiveBayes'].values
modelo_negativo = LinearRegression()
modelo_negativo.fit(X_negativo, y_negativo)

# Ajustar un modelo de regresión lineal a los datos históricos de
sentimiento positivo
X_positivo = np.arange(len(df_positivo_agrupado)).reshape(-1, 1)
y_positivo = df_positivo_agrupado['Probabilidad_por_NaiveBayes'].values
modelo_positivo = LinearRegression()
modelo_positivo.fit(X_positivo, y_positivo)

# Predecir los próximos 20 días para sentimiento negativo
proximos_dias_negativo =
pd.date_range(start=df_negativo_agrupado['Fecha'].iloc[-1] +
pd.Timedelta(days=1), periods=20, freq='D')
proximos_indices_negativo = np.arange(len(df_negativo_agrupado),
len(df_negativo_agrupado) + 20).reshape(-1, 1)
predicciones_negativo = modelo_negativo.predict(proximos_indices_negativo)
predicciones_df_negativo = pd.DataFrame({'Fecha': proximos_dias_negativo,
'prediccion_probabilidad_negativo': predicciones_negativo})

# Predecir los próximos 20 días para sentimiento positivo
proximos_dias_positivo =
pd.date_range(start=df_positivo_agrupado['Fecha'].iloc[-1] +
pd.Timedelta(days=1), periods=20, freq='D')
proximos_indices_positivo = np.arange(len(df_positivo_agrupado),
len(df_positivo_agrupado) + 20).reshape(-1, 1)
predicciones_positivo = modelo_positivo.predict(proximos_indices_positivo)
predicciones_df_positivo = pd.DataFrame({'Fecha': proximos_dias_positivo,
'prediccion_probabilidad_positivo': predicciones_positivo})

# Graficar los datos históricos y las predicciones para sentimiento negativo
plt.figure(figsize=(10, 6))
plt.plot(df_negativo_agrupado['Fecha'],
df_negativo_agrupado['Probabilidad_por_NaiveBayes'], label='Negativo - Datos
Históricos', marker='o', color='red')
```

```
plt.plot(predicciones_df_negativo['Fecha'],
predicciones_df_negativo['prediccion_probabilidad_negativo'],
label='Negativo - Predicción', linestyle='--', marker='o', color='darkred')

# Graficar los datos históricos y las predicciones para sentimiento positivo
plt.plot(df_positivo_agrupado['Fecha'],
df_positivo_agrupado['Probabilidad_por_NaiveBayes'], label='Positivo - Datos
Históricos', marker='o', color='green')
plt.plot(predicciones_df_positivo['Fecha'],
predicciones_df_positivo['prediccion_probabilidad_positivo'],
label='Positivo - Predicción', linestyle='--', marker='o',
color='darkgreen')

#plt.title('Predicción de Probabilidad por Sentimiento de Naive Bayes
(Claudia Sheinbaum)')
plt.title('Predicción de Probabilidad por Sentimiento de Naive Bayes
(Xochitl Gálvez)')
plt.xlabel('Fecha')
plt.ylabel('Probabilidad Promedio')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Conclusiones

El análisis predictivo de opiniones en redes sociales, como el realizado en este manual técnico para las elecciones presidenciales de 2024 en México, representa una herramienta poderosa para comprender y anticipar tendencias en la opinión pública. A lo largo de este manual, se han presentado técnicas y herramientas clave para llevar a cabo este tipo de análisis de manera efectiva. La recolección de datos, la limpieza y procesamiento de textos, y la aplicación de algoritmos de análisis de sentimientos son pasos fundamentales en este proceso. La utilización de herramientas como Botster para la recolección de comentarios en YouTube y librerías de Python como Numpy, Matplotlib, Unidecode, Spacy y Sklearn para el procesamiento de datos y la implementación de modelos de aprendizaje automático ha demostrado ser efectiva en la predicción de tendencias de opinión.

Es importante destacar que el análisis predictivo de opiniones en redes sociales no solo proporciona información útil para campañas políticas, sino que también puede ser aplicado en otros contextos, como estudios de mercado, análisis de satisfacción del cliente y detección de tendencias en temas específicos.

En resumen, este manual técnico proporciona una base sólida para aquellos interesados en utilizar el análisis predictivo de opiniones en redes sociales como una herramienta para comprender mejor la opinión pública y anticipar tendencias en diversos ámbitos. Su aplicación puede ser de gran utilidad en la toma de decisiones informadas y estratégicas en diferentes áreas.

Referencias

Ortega, C. (2023, May 19). *Minería de datos: Qué es, ventajas, técnicas y cómo realizarla*. QuestionPro. https://www.questionpro.com/blog/es/mineria-de-datos/

¿Qué es la minería de datos? | Definición, importancia y tipos | SAP Insights. (2017). SAP. https://www.sap.com/latinamerica/products/technology-platform/hana/what-is-data-mining.html

Qué es la minería de datos: conceptos, técnicas y ejemplos. (2022, September 19). Hubspot.es. https://blog.hubspot.es/marketing/mineria-datos

¿Qué es la minería de datos? | IBM. (2023, July 18). Ibm.com. https://www.ibm.com/es-es/topics/data-mining

t-Test, Chi-Square, ANOVA, Regression, Correlation... (n.d.). Datatab.es. https://datatab.es/tutorial/linear-regression

Estadística, P. y. (2023, March 9). *▷ Regresión lineal*. Probabilidad Y Estadística. https://www.probabilidadyestadistica.net/regresion-lineal/

Botster – data extraction, monitoring and automation. (2016). Botster.io. https://botster.io/